# Java & Spring Framework

## Complete Study Guide

| Topics Covered | 50+ Core Java & Spring Concepts |
| --- | --- |
| **Includes** | String, Arrays, Collections, OOP, JDBC, Exceptions, Annotations, Lambda, Streams, Optional, Spring Core, AOP, MVC, REST API, Stereotype Annotations |
| **Level** | Beginner to Advanced |

## Table of Contents

# Chapter 1: Core Java Fundamentals

## 1.1 String in Java

A **String** is a sequence of characters. In Java, String is an **Object** (from `java.lang.String`), not a primitive type.

### Ways to Create a String

- **String literal**: `String s = "cts";` — stored in SCP, reuses existing object
- **new keyword**: `String s = new String("cts");` — always creates new Heap object
- **StringBuffer**: `StringBuffer sb = new StringBuffer("cts");` — mutable, thread-safe
- **StringBuilder**: `StringBuilder sb = new StringBuilder("cts");` — mutable, faster

| Type | Mutable? | Thread-Safe? | Storage | Speed |
|---|---|---|---|---|
| String | No (Immutable) | Yes | SCP / Heap | Fast |
| StringBuffer | Yes | Yes | Heap | Slow |
| StringBuilder | Yes | No | Heap | Fastest |

■ ***SCP = String Constant Pool. Literal strings are reused — new String() always creates a fresh Heap object.***

## 1.2 Arrays

An **Array** stores multiple values of the **same data type** with a **fixed size**, index-based from 0.

### Arrays Utility Class — Key Methods

| Method | Purpose | Example |
|---|---|---|
| Arrays.sort(arr) | Sort ascending | Arrays.sort(arr) |
| Arrays.toString(arr) | Print 1D array | Arrays.toString(arr) |
| Arrays.binarySearch(arr,key) | Search (sorted array) | Arrays.binarySearch(arr, 30) |
| Arrays.equals(a, b) | Compare two arrays | Arrays.equals(a, b) |
| Arrays.fill(arr, val) | Fill with value | Arrays.fill(arr, 7) |
| Arrays.copyOf(arr, len) | Copy first n elements | Arrays.copyOf(arr, 3) |
| Arrays.deepToString(arr) | Print 2D array | Arrays.deepToString(matrix) |

## 1.3 Classes & Objects

A **Class** is a blueprint/template. An **Object** is a real instance created from a class using the new keyword.

### Members of a Class

- **Static Member Variable** — ONE copy shared by all objects, stored in Method Area
- **Instance Member Variable** — each object gets its own copy, stored in Heap
- **Static Member Function** — called without object, cannot access instance vars
- **Instance Member Function** — needs an object, can access all vars
- **Constructor** — Default, Parameterized, Copy
- **Static Block** — runs ONCE when class loads, before main()

## Execution Order

```
Static Block → main() → Constructor → Instance Methods
```

# Chapter 2: Object-Oriented Programming

## 2.1 Polymorphism

**Polymorphism** means 'one thing, many forms'. Java has two types:

| Type | How | Binding | Key Rule |
|---|---|---|---|
| Compile-Time (Overloading) | Same method name, different parameters in same class | Early / Static Binding | Resolved at compile time |
| Runtime (Overriding) | Same method name + params in parent & child class | Late / Dynamic Binding | Resolved at runtime based on actual object |

### Object Casting

- **Upcasting** — Child → Parent. Automatic, always safe. Access only parent methods.
- **Downcasting** — Parent → Child. Manual cast needed. Use `instanceof` check to avoid ClassCastException.

## 2.2 Abstraction

| Feature | Abstract Class | Interface |
|---|---|---|
| Object creation | Cannot create | Cannot create |
| Methods | Abstract + Normal | Abstract (default/static from Java 8) |
| Variables | Any type | public static final only |
| Constructor | Yes | No |
| Multiple inheritance | No | Yes |
| IS-A possible? | Yes (via extends) | Yes (via implements) |
| HAS-A possible? | No (cannot instantiate) | No (cannot instantiate) |
| Use when | Related classes sharing code | Unrelated classes, same behaviour |

## 2.3 Inheritance

| Type | Description | Note |
|---|---|---|
| Single | One child, one parent | Dog extends Animal |
| Multilevel | Chain: A → B → C | Puppy extends Dog extends Animal |
| Hierarchical | One parent, many children | Dog, Cat, Bird all extend Animal |
| Multiple | One child, many parents | Classes: NOT allowed. Interfaces: OK |
| Hybrid | Combination of above | Only via interfaces |

## HAS-A (Association)

- **Aggregation (Weak)** — Student HAS-A Address. Both can exist independently.
- **Composition (Strong)** — House HAS-A Room. Room cannot exist without House.

## final class — IS-A vs HAS-A

- **IS-A NOT possible** — final class cannot be inherited (compiler error)
- **HAS-A IS possible** — final class CAN create objects, so aggregation works

# Chapter 3: Modifiers & Memory

## 3.1 Non-Access Modifiers

| Modifier | Level | Effect |
|---|---|---|
| final (class) | Class | Cannot be inherited |
| final (variable) | Variable | Cannot be reassigned — becomes constant |
| final (method) | Method | Cannot be overridden |
| static | Variable/Method | One copy shared; call without object |
| synchronized | Method | One thread at a time — thread-safe |
| transient | Variable | Skipped during serialization |
| volatile | Variable | Always read from main memory |
| abstract | Class/Method | Cannot instantiate; must be implemented |

## 3.2 JVM Memory & Object Creation

| Memory Area | What is Stored |
|---|---|
| Method Area | Class definitions, static variables, static blocks, static methods |
| Heap Memory | Actual objects, instance variables, String Constant Pool (SCP) |
| Stack Memory | Reference variables, local variables, method call frames |

### Primitive vs Reference Types

| Feature | Primitive | Reference |
|---|---|---|
| Stores | Actual value | Address of object |
| Memory | Stack only | Stack (ref) + Heap (object) |
| Default value | 0 / false / '\u0000' | null |
| Can be null? | No | Yes |
| Assignment | Copies value | Copies address |
| Examples | int, double, char | String, Array, Object |

# Chapter 4: Collections Framework

## 4.1 Collection Hierarchy

```
Collection (I) ■■■ List (I) → ArrayList, LinkedList, Vector ■■■ Set (I) → HashSet,
LinkedHashSet, TreeSet ■■■ Queue(I) → PriorityQueue, LinkedList Map (I) → HashMap,
LinkedHashMap, TreeMap, Hashtable
```

## 4.2 When to Use Which Collection

| Collection | Ordered? | Duplicates? | Null? | DS Used | Best Use Case |
|---|---|---|---|---|---|
| ArrayList | Insertion order | Yes | Multiple | Dynamic Array | Fast reads, ordered list |
| LinkedList | Insertion order | Yes | Multiple | Doubly Linked List | Frequent add/remove at ends |
| Vector | Insertion order | Yes | Multiple | Dynamic Array | Thread-safe list (legacy) |
| HashSet | No order | No | One null | Hash Table | Unique items, fastest |
| LinkedHashSet | Insertion order | No | One null | HashTable+Linked List | Unique + insertion order |
| TreeSet | Sorted | No | No | Red-Black Tree | Sorted unique items |
| HashMap | No order | No dup keys | One null key | Hash Table | Fast key-value lookup |
| LinkedHashMap | Insertion order | No dup keys | One null key | HT+LL | Ordered key-value |
| TreeMap | Sorted by key | No dup keys | No null key | RB Tree | Sorted key-value pairs |

# Chapter 5: JDBC

## 5.1 Steps to Connect to Database

- **Step 1 — Load Driver**: `Class.forName("com.mysql.cj.jdbc.Driver");`
- **Step 2 — Get Connection**: `DriverManager.getConnection(url, user, pass);`
- **Step 3 — Create Statement**: Statement / PreparedStatement / CallableStatement
- **Step 4 — Execute**: execute() / executeUpdate() / executeQuery()
- **Step 5 — Process ResultSet**: `while(rs.next()) { rs.getString("col"); }`
- **Step 6 — Close**: rs → stmt → con (in finally block)

| Statement Type | Used For | Safe? | Speed |
|---|---|---|---|
| Statement | Simple static SQL | No (SQL Injection risk) | Slow |
| PreparedStatement | Parameterised SQL (?) | Yes | Fast |
| CallableStatement | Stored Procedures | Yes | Fast |

| Execute Method | Returns | Used For |
|---|---|---|
| execute() | boolean | Any SQL (DDL + DML + SELECT) |
| executeUpdate() | int (rows affected) | INSERT / UPDATE / DELETE / DDL |
| executeQuery() | ResultSet | SELECT only |

# Chapter 6: Exception Handling

## 6.1 Exception Hierarchy

```
Throwable ■■■ Error (JVM level — unrecoverable: OutOfMemoryError) ■■■ Exception ■■■ Checked Exception (compile time: IOException, SQLException) ■■■ RuntimeException (unchecked: NPE, ArrayIndexOutOfBounds)
```

| Feature | Checked | Unchecked |
|---|---|---|
| Detected | Compile time | Runtime |
| Compiler forces? | Yes | No |
| Cause | External resources | Programming mistakes |
| Examples | IOException, SQLException | NullPointerException, ArithmeticException |
| throws needed? | Mandatory | Optional |

## 6.2 Handling & Propagation

```
try { // risky code } catch (SpecificException e) { // handle specific } catch (Exception e) { // handle general (always last) } finally { // ALWAYS runs — close resources here }
```

- **throw** — actually throws an exception: `throw new MyException();`
- **throws** — declares exception in method signature: `void m() throws IOException`
- **Custom Checked**: extend `Exception`
- **Custom Unchecked**: extend `RuntimeException`

# Chapter 7: Functional Programming

## 7.1 Functional Interface

A **Functional Interface** has exactly ONE abstract method. Used with Lambda expressions.

| Interface | Method | Input | Output | Use For |
|---|---|---|---|---|
| Predicate<T> | test(T) | T | boolean | Test/filter conditions |
| Consumer<T> | accept(T) | T | void | Consume/print data |
| Supplier<T> | get() | nothing | T | Create/supply values |
| Function<T,R> | apply(T) | T | R | Transform/convert |

## 7.2 Lambda Expression

```
(parameters) -> { body } // Examples: () -> System.out.println("No params") n -> n * n // single param, expression (a, b) -> a + b // two params (a, b) -> { int s = a+b; return s; } // block body
```

## 7.3 Stream API

Stream API processes data in a **Source** → **Intermediate Operations** → **Terminal Operation** pipeline.

| Category | Operation | Returns | Purpose |
|---|---|---|---|
| Source | collection.stream() / Arrays.stream() | Stream | Create stream |
| Intermediate | filter(predicate) | Stream | Keep matching elements |
| Intermediate | map(function) | Stream | Transform each element |
| Intermediate | sorted() | Stream | Sort elements |
| Intermediate | distinct() | Stream | Remove duplicates |
| Terminal | forEach(consumer) | void | Process each element |
| Terminal | collect(Collectors.toList()) | Collection | Gather results |
| Terminal | findFirst() | Optional | First matching element |
| Terminal | count() | long | Count elements |
| Terminal | reduce() | Optional | Combine all elements |

## 7.4 Optional

| Method | Returns | Use Case |
|---|---|---|
| Optional.of(val) | Optional | Non-null value — NPE if null |

| | | |
|---|---|---|
| Optional.ofNullable(val) | Optional | May be null — returns empty if null |
| Optional.empty() | Optional | Create empty Optional |
| isPresent() | boolean | Check if value exists (Java 8) |
| isEmpty() | boolean | Check if empty (Java 11) |
| get() | T | Get value — throws if empty (risky!) |
| orElse(default) | T | Value or constant default |
| orElseGet(supplier) | T | Value or computed default (lazy) |
| orElseThrow(supplier) | T | Value or throw custom exception |
| ifPresent(consumer) | void | Run action only if value present |
| map(function) | Optional | Transform value if present |

# Chapter 8: Spring Framework Core

## 8.1 Spring Core Concepts

| Concept | Meaning | Benefit |
|---|---|---|
| IoC (Inversion of Control) | Spring creates and manages objects (beans) instead of programmer | No manual new keyword; Spring wires everything |
| DI (Dependency Injection) | Spring automatically provides dependencies to classes | Loose coupling; easy to test with mocks |
| AOP (Aspect-Oriented) | Separate cross-cutting concerns (logging, security, tx) | Clean business logic; concerns written once |

## 8.2 IoC Container Types

| Container | Loading | Features | Use |
|---|---|---|---|
| BeanFactory | LAZY — on demand | Basic DI only | Limited-memory / simple apps |
| ApplicationContext | EAGER — at startup | AOP, Events, i18n, @PostConstruct | All real-world apps ✓ |
| ClassPathXmlApplicationContext | EAGER | Reads beans.xml from classpath | XML config |
| AnnotationConfigApplicationContext | EAGER | Reads @Configuration Java class | Java config (modern) |
| WebApplicationContext | EAGER | Web scopes: request, session | Web / Spring Boot apps |

## 8.3 Bean Configuration Types

| Type | Config | Java Classes | Context Class |
|---|---|---|---|
| 8.1 XML + XML | beans.xml with <bean> tags | No annotations | ClassPathXmlApplicationContext |
| 8.2 XML + Annotation | beans.xml with <context:component-scan> | @Component etc. | ClassPathXmlApplicationContext |
| 8.3a Java + @Bean | @Configuration class with @Bean methods | No annotations needed | AnnotationConfigApplicationContext |
| 8.3b Java + @ComponentScan | @Configuration + @ComponentScan | @Component, @Service... | AnnotationConfigApplicationContext |
| Spring Boot | @SpringBootApplication | @Component, @Service... | Auto (WebApplicationContext) |

## 8.4 Dependency Injection Types

| Type | How | Immutable? | Testable? | Recommended? |
|---|---|---|---|---|
| Constructor Injection | @Autowired on constructor | Yes (final) | Easiest | YES ✓ |
| Setter Injection | @Autowired on setter | No | Easy | For optional deps |
| Field Injection | @Autowired on field | No | Harder | Avoid |

# Chapter 9: Spring Beans

## 9.1 Bean Scope

| Scope | Instances | Created When | Destroyed When | Use For |
|---|---|---|---|---|
| singleton | ONE per container | App startup | App shutdown | Services, Repos (DEFAULT) |
| prototype | NEW each getBean() | On request | Not by Spring | Stateful beans |
| request | ONE per HTTP request | Request starts | Request ends | Request tracking (web) |
| session | ONE per HTTP session | Session starts | Session ends | Shopping cart (web) |
| application | ONE per ServletCtx | App startup | App shutdown | Global config (web) |

## 9.2 Bean Lifecycle

```
1. Constructor called 2. Dependencies injected (@Autowired) 3. @PostConstruct ← programmer's init code (runs 1st) 4. afterPropertiesSet() ← InitializingBean (runs 2nd) 5. init-method ← XML/@Bean config (runs 3rd) 6. Bean READY — application uses it 7. @PreDestroy ← programmer's cleanup (runs 1st) 8. DisposableBean.destroy() ← DisposableBean (runs 2nd) 9. destroy-method ← XML/@Bean config (runs 3rd)
```

## 9.3 Ambiguity — @Primary vs @Qualifier

| Annotation | Where Used | Priority | Purpose |
|---|---|---|---|
| @Primary | On bean class | 2nd | Default bean when no @Qualifier |
| @Qualifier | At injection point | 1st | Exact bean by name — overrides @Primary |

■ ***Spring resolution order: @Qualifier (wins) → @Primary → field name match → Exception***

# Chapter 10: Spring AOP

## 10.1 AOP Key Terms

| Term | What It Is | Example |
|---|---|---|
| JoinPoint | Point where advice CAN run (any method) | Any method in service layer |
| Pointcut | Expression filtering which JoinPoints | execution(* com.app.service.*.*(..)) |
| Advice | The actual code that runs | @Before, @After, @Around |
| Aspect | Class containing cross-cutting logic | @Aspect @Component LoggingAspect |
| Weaving | Applying aspects to target objects | Spring does this via Proxy |

## 10.2 Advice Types

| Advice | When | Key Use Case |
|---|---|---|
| @Before | Before method executes | Security check, logging entry |
| @After | After method (always — like finally) | Audit, release resources |
| @AfterReturning | After method returns successfully | Log result, cache value |
| @AfterThrowing | After method throws exception | Log exception, send alert |
| @Around | Before AND after (most powerful) | Timing, transactions |

## 10.3 AOP Proxy

| Proxy Type | When Used | How | Spring Boot Default? |
|---|---|---|---|
| JDK Dynamic Proxy | Bean implements interface | Creates proxy implementing same interface | No |
| CGLIB Proxy | Bean has no interface | Subclasses the bean class at runtime | Yes ✓ |

■ ***Self-invocation (this.method()) bypasses proxy — aspects won't run! Inject self-reference to fix.***

## 10.4 Cross-Cutting Concerns

- **Logging** — @Around advice logs method entry, exit, time, exceptions for all service methods
- **Security** — @Before advice checks authentication/authorisation before any method
- **Transaction** — @Around advice begins/commits/rollbacks DB transaction (@Transactional uses AOP internally)

# Chapter 11: Spring MVC & REST API

## 11.1 MVC vs REST Comparison

| Feature | Spring MVC | Spring REST |
|---|---|---|
| Annotation | @Controller | @RestController |
| Returns | HTML page (View name) | JSON / XML data |
| Client | Browser only | Browser, Mobile, Any client |
| Template | Thymeleaf / JSP | None needed |
| Input binding | @ModelAttribute (form data) | @RequestBody (JSON) |
| Output conversion | ViewResolver + Thymeleaf | HttpMessageConverter (Jackson) |
| Return type | String (view name) | ResponseEntity<T> |
| Error handling | @ControllerAdvice | @RestControllerAdvice |
| Use case | Traditional web apps | Microservices, Mobile, SPA |

## 11.2 MVC Flow

```
Browser → DispatcherServlet → HandlerMapping → @Controller → Service → Repository → DB →
model.addAttribute() → return "view/name" → ViewResolver → Thymeleaf renders HTML → Browser
```

## 11.3 REST Flow

```
Client + JSON body → DispatcherServlet → HandlerMapping → @RestController → Jackson converts
JSON → Java object (@RequestBody) → Service → Repository → DB → return
ResponseEntity.status(201).body(savedObject) → Jackson converts Java → JSON → Client
```

## 11.4 HTTP Mapping Annotations

| Annotation | HTTP Method | Use Case |
|---|---|---|
| @GetMapping | GET | Fetch / display data |
| @PostMapping | POST | Create / submit form |
| @PutMapping | PUT | Full update |
| @PatchMapping | PATCH | Partial update |
| @DeleteMapping | DELETE | Delete resource |

## 11.5 @Controller vs @RestController

| Feature | @Controller | @RestController |
| --- | --- | --- |
| Returns | View name (String) | Data (Object / ResponseEntity) |
| @ResponseBody | Must add on each method | Applied to ALL methods automatically |
| Equals | @Controller only | @Controller + @ResponseBody |
| Template needed? | Yes — Thymeleaf/JSP | No |
| Use for | Server-rendered HTML apps | REST APIs, Microservices |

# Chapter 12: Stereotype Annotations

## 12.1 Annotation Hierarchy

```
@Component (ROOT - generic bean) ■■■ @Repository → Data Access Layer + Exception Translation
■■■ @Service → Business Logic Layer ■■■ @Controller → Web Layer (MVC) - returns view names ■■■
@RestController = @Controller + @ResponseBody
```

## 12.2 Full Comparison

| Annotation | Layer | Returns | Special Feature | Example |
|---|---|---|---|---|
| @Component | Any / Generic | Anything | Generic Spring bean | EmailService, CacheService |
| @Repository | Data Access | Anything | Translates DB exceptions automatically | StudentRepository |
| @Service | Business Logic | Anything | Business rules & processing | StudentService, OrderService |
| @Controller | Web (MVC) | View name | Handles HTTP, finds template | StudentController |
| @RestController | Web (REST) | JSON/XML | @ResponseBody on all methods | StudentRestController |

## 12.3 Loose Coupling with Spring

Spring achieves loose coupling through **Aggregation via Interface** + **Dependency Injection**:

- **Step 1** — Define an Interface (e.g., Database, PaymentService)
- **Step 2** — Create multiple implementations (@Service classes)
- **Step 3** — Declare interface type as field in dependent class (HAS-A via interface = Aggregation)
- **Step 4** — Use @Autowired + @Qualifier to inject specific implementation
- **Step 5** — IoC Container creates all beans and wires them together

■ ***Change implementation? Just change @Qualifier — business logic class is NEVER modified!***

# Quick Reference Card

## Key Annotations Summary

| Annotation | Purpose |
|---|---|
| @SpringBootApplication | Start Spring Boot — includes @ComponentScan + @Configuration + @EnableAutoConfiguration |
| @Component | Generic Spring-managed bean |
| @Service | Business logic bean |
| @Repository | Data access bean + exception translation |
| @Controller | MVC web controller — returns view names |
| @RestController | REST controller — returns JSON (= @Controller + @ResponseBody) |
| @Autowired | Inject dependency (field / constructor / setter) |
| @Qualifier("name") | Specify exact bean to inject when multiple exist |
| @Primary | Default bean when no @Qualifier specified |
| @Scope("prototype") | New bean instance every time requested |
| @PostConstruct | Run after dependency injection — initialization |
| @PreDestroy | Run before bean destroyed — cleanup |
| @Transactional | Spring manages DB transaction via AOP |
| @Aspect | Marks class as AOP aspect |
| @Before / @After | Run advice before / after matched method |
| @Around | Wrap method — before + after + control execution |
| @RequestMapping | Map URL to controller class or method |
| @GetMapping | Map HTTP GET to method |
| @PostMapping | Map HTTP POST to method |
| @PathVariable | Extract value from URL path /students/{id} |
| @RequestParam | Extract query parameter ?name=value |
| @RequestBody | Convert JSON request body to Java object |
| @ResponseBody | Convert return value to JSON response |
| @Valid | Trigger Bean Validation on annotated param |
| @Entity | Map class to database table (JPA) |
| @FunctionalInterface | Enforce exactly one abstract method |

## Common Interview Quick-Fire Answers

| Question | Answer |
|---|---|
| String immutable — why? | Value cannot change after creation; any modification creates a new object |
| SCP vs Heap for String? | Literals → SCP (reused). new String() → always new Heap object |
| ArrayList vs LinkedList? | ArrayList: fast get O(1). LinkedList: fast add/remove at ends O(1) |
| HashSet vs TreeSet? | HashSet: O(1) unordered. TreeSet: O(log n) sorted |
| Checked vs Unchecked? | Checked: compiler forces handling (IO, SQL). Unchecked: runtime bugs (NPE) |
| IoC vs DI? | IoC = control given to Spring. DI = Spring injects dependencies automatically |
| @Primary vs @Qualifier? | @Qualifier always wins (specific). @Primary is default fallback |
| Singleton vs Prototype? | Singleton: one per container (default). Prototype: new each time |
| @Controller vs @RestController? | @RestController = @Controller + @ResponseBody (returns JSON not views) |
| AOP Proxy types? | JDK Dynamic (interface present). CGLIB (no interface — Spring Boot default) |
| @PostConstruct vs init-method? | @PostConstruct runs first, then afterPropertiesSet(), then init-method |
| final class — HAS-A or IS-A? | IS-A impossible (cannot extend). HAS-A possible (can create objects) |
| Abstract — HAS-A or IS-A? | IS-A possible (child extends). HAS-A impossible (cannot instantiate) |
| BeanFactory vs ApplicationContext? | BeanFactory: lazy, basic. ApplicationContext: eager, full features — always use AppCtx |